



Professional Documentation for

yugipedia_card_details_scraper.py **v3**

Overview

yugipedia_card_details_scraper.py is a production-grade Python script designed to scrape comprehensive Yu-Gi-Oh! card details from Yugipedia (<https://yugipedia.com>) via its web interface. The script employs concurrent processing with intelligent rate limiting and enhanced retry logic to efficiently retrieve detailed information for over 13,000 cards while fully respecting Wikimedia Foundation policies and handling Cloudflare protection mechanisms.

Version History

- **v1.0:** Initial release with basic scraping functionality
- **v2.0:** Added bug fixes for ATK/DEF strings, 10000 Secret Rare rarity, and Link markers
- **v3.0:** Enhanced retry logic with progressive backoff and intelligent error detection for network resilience

Purpose

To extract complete card information including:

- Basic metadata (ID with leading zeros, name, type, properties)
- Monster-specific attributes (ATK/DEF, level, rank, link markers)
- Card effects and descriptions (including Pendulum effects)
- Archetype associations
- Complete printing history across all card sets with rarity information

Technical Requirements

Dependencies

```
pip install cloudscraper beautifulsoup4 lxml
```

- **Python:** 3.8 or higher
- **cloudscraper:** 1.2.58+ (Bypasses Cloudflare JavaScript challenges)

- **beautifulsoup4**: 4.9.0+ (HTML parsing and data extraction)
- **lxml**: 4.6.0+ (Optional but recommended for faster HTML parsing)

Input File Format

Required: `yugipedia_passcode_list.json`

Structure:

```
[
  {
    "name": "Blue-Eyes White Dragon",
    "card_type": "Monster Card",
    "password": "89631139",
    "url": "https://yugipedia.com/wiki/Blue-Eyes_White_Dragon"
  },
  {
    "name": "Parallel Teleport",
    "card_type": "Spell Card",
    "password": "00000483",
    "url": "https://yugipedia.com/wiki/Parallel_Teleport"
  }
]
```

Required Fields:

- `name` (string): Card name
- `card_type` (string): "Monster Card", "Spell Card", or "Trap Card"
- `password` (string): 8-digit passcode with leading zeros
- `url` (string): Full URL to card's Yugipedia page

Output Files

1. `yugipedia_all_cards.json`: Successfully scraped cards with complete details
2. `yugipedia_rejected_cards.json`: Failed cards with rejection reasons, timestamps, and retry information

Architecture and Design

Concurrent Processing Model

The script uses **ThreadPoolExecutor** with the following characteristics:

- **Workers**: 8 concurrent threads (configurable)
- **Rate Limit**: 3 requests per second (global, thread-safe)
- **Approach**: Token bucket algorithm for precise rate limiting

- **Scraper Distribution:** Round-robin assignment of CloudScraper instances to workers

Enhanced Retry Logic (v3)

Configuration:

```
MAX_RETRIES = 5           # Increased from 3
REQUEST_TIMEOUT = 60      # Increased from 30 seconds
RETRY_DELAYS = [3, 5, 10, 15, 20] # Progressive delays in seconds
```

Retry Strategy:

1. **Attempt 1:** Immediate request
2. **Attempt 2:** Wait 3-5 seconds (3s base + 0-2s jitter)
3. **Attempt 3:** Wait 5-7 seconds (5s base + 0-2s jitter)
4. **Attempt 4:** Wait 10-12 seconds (10s base + 0-2s jitter)
5. **Attempt 5:** Wait 15-17 seconds (15s base + 0-2s jitter)
6. **Attempt 6:** Wait 20-22 seconds (20s base + 0-2s jitter)

Intelligent Error Detection:

The script automatically retries on these transient errors:

- ✓ **502 Bad Gateway** - Server temporarily down
- ✓ **503 Service Unavailable** - Server overloaded
- ✓ **500 Internal Server Error** - Temporary server issue
- ✓ **504 Gateway Timeout** - Server took too long
- ✓ **ReadTimeout** - Server response timeout
- ✓ **ConnectTimeout** - Connection establishment timeout
- ✓ **ConnectionError** - Network connection issues

Non-Retryable Errors:

- 404 Not Found (card doesn't exist)
- 403 Forbidden (blocked access)
- Password mismatch (data integrity issue)
- Parsing errors (invalid HTML structure)

Thread-Safe Rate Limiting

```
class RateLimiter:
    """Ensures no more than 3 req/sec across all threads"""
```

Uses mutex locks to coordinate request timing globally, preventing any thread from exceeding the rate limit even with concurrent execution.

Cloudflare Bypass

Employs `cloudscraper` library which:

- Automatically solves JavaScript challenges
- Handles cookie management
- Mimics legitimate browser behavior
- Maintains User-Agent header for Wikimedia compliance
- Adapts to Cloudflare updates automatically

Key Features

1. Card Type Handlers

Three specialized parsing functions:

- `parse_monster_card()`: Handles Monster Cards with all attributes
- `parse_spell_card()`: Processes Spell Cards with properties
- `parse_trap_card()`: Processes Trap Cards with properties

2. Data Extraction Capabilities

Monster Cards:

- **Typeline parsing:** Aqua, Beast, Dragon, etc. (25 types)
- **Mechanic detection:** Normal, Fusion, Synchro, Xyz, Pendulum, Link (12 mechanics)
- **Level/Rank extraction:** Integer values with fallback
- **ATK/DEF values:** Handles both integers (2500) and strings ("?", "??")
- **Link markers:** Extracts only actual markers present on card (not all 8 positions)
- **Pendulum scale:** For Pendulum monsters only
- **Summoning conditions:** For Extra Deck monsters (Fusion, Synchro, Xyz, Link)

Spell/Trap Cards:

- **Property extraction:** Normal, Continuous, Quick-Play, Counter, Equip, Field
- **Effect descriptions:** Complete text with line breaks preserved
- **Archetype associations:** Multiple archetypes supported

All Cards:

- **Complete set history:** All printings with dates, codes, and rarities

- **Rarity code mapping:** 18 different rarities including 10000 Secret Rare
- **Password verification:** Cross-checks password on page vs input
- **Archetype/series detection:** Multiple archetypes per card

3. Robust Error Handling

Network-Level:

- Retry logic with exponential backoff (5 attempts)
- Timeout handling (60 seconds per request)
- Random jitter to prevent thundering herd
- Intelligent error type detection

Data-Level:

- Password mismatch detection
- Missing field handling (graceful degradation)
- Type conversion with fallbacks
- HTML structure validation

Process-Level:

- Thread-safe operations
- Progress tracking with ETA
- Intermediate failure recovery
- Comprehensive error logging

Configuration Options

Performance Tuning

```
# Core settings
MAX_WORKERS = 8           # Concurrent threads (safe: 5-10)
REQUESTS_PER_SECOND = 3   # Rate limit (safe: 2-4)
MIN_REQUEST_INTERVAL = 1.0 / REQUESTS_PER_SECOND

# Retry settings (v3)
MAX_RETRIES = 5           # Retry attempts (recommended: 5)
REQUEST_TIMEOUT = 60      # Timeout in seconds (recommended: 60)
RETRY_DELAYS = [3, 5, 10, 15, 20] # Progressive delays
```

Performance Matrix:

Workers	Req/Sec	Timeout	Retries	Time (13,663 cards)	Risk Level	Rejections
5	2	30s	3	~114 min	None	~50

Workers	Req/Sec	Timeout	Retries	Time (13,663 cards)	Risk Level	Rejections
8	3	60s	5	~76 min	Low	<5
10	4	60s	5	~57 min	Medium	<5
15	5	60s	5	~45 min	High	<5

Recommended: 8 workers, 3 req/sec, 60s timeout, 5 retries (current default)

User-Agent Configuration

```
USER_AGENT = 'YugipediaCardBot/1.0 (https://github.com/ege24; ege24@gmail.com) requests'
```

Components:

- Bot name and version: YugipediaCardBot/1.0
- Contact information: GitHub + email
- Library identification: requests/2.31.0

Wikimedia Foundation Policy Compliance

User-Agent Policy

Reference: https://foundation.wikimedia.org/wiki/Policy:Wikimedia_Foundation_User-Agent_Policy

Compliance Measures:

1. ✓ Descriptive User-Agent with contact information
2. ✓ Includes "bot" keyword for traffic classification
3. ✓ Rate limiting (3 req/sec is conservative)
4. ✓ Polite request intervals (minimum 0.33s)
5. ✓ No browser User-Agent impersonation
6. ✓ Retry logic with progressive backoff delays
7. ✓ Respects server errors (502, 503) with increased delays

Best Practices

- Script waits minimum 0.33 seconds between any two requests
- Uses proper HTML endpoints (not aggressive scraping)
- Handles errors gracefully without overwhelming server
- Progressive backoff gives server time to recover
- Random jitter prevents synchronized retry storms

- Can be safely used repeatedly without causing issues

Function Reference

Core Functions

`create_scraper()`

Creates CloudScraper instance configured to bypass Cloudflare.

Returns: Configured scraper object with User-Agent

Example:

```
scraper = create_scraper()
```

`fetch_page(scraper, url, retries=5)`

Fetches HTML content from URL with enhanced retry logic and rate limiting.

Parameters:

- `scraper`: CloudScraper instance
- `url`: Target URL
- `retries`: Number of retry attempts (default: 5)

Returns: (`html_text`, `error`) tuple

Error Handling:

- Automatically retries on 502, 503, 500, 504, timeouts
- Uses progressive delays: [3s, 5s, 10s, 15s, 20s]
- Adds random jitter (0-2 seconds) to each delay
- Returns detailed error message on final failure

Example:

```
html, error = fetch_page(scraper, url)
if error:
    print(f"Failed to fetch: {error}")
else:
    soup = BeautifulSoup(html, 'html.parser')
```

```
extract_atk_def(soup, is_link=False)
```

Extracts ATK/DEF values, handling both integers and strings.

Parameters:

- `soup`: BeautifulSoup object
- `is_link`: Boolean indicating Link monster

Returns: Dictionary with 'atk', 'def', or 'link_rating'

Special Cases:

- Regular cards: Returns integers for ATK/DEF
- Unknown values: Returns strings ("?", "???", "X000")
- Link monsters: Returns ATK (int/string) and link_rating (int)

Example:

```
atk_def = extract_atk_def(soup, is_link=False)
# Returns: {'atk': 2500, 'def': 2000}
# Or: {'atk': '?', 'def': '?'}
```

```
extract_link_markers(soup)
```

Extracts actual link markers present on the card.

Logic: Reads text list (hlist) instead of images to determine active markers

Returns: List of markers (e.g., ["Top", "Right", "Bottom"]) or None

Mapping:

- Top-Left → Top-Left
- Top-Center → Top
- Top-Right → Top-Right
- Middle-Left → Left
- Middle-Right → Right
- Bottom-Left → Bottom-Left
- Bottom-Center → Bottom
- Bottom-Right → Bottom-Right

Example:

```
markers = extract_link_markers(soup)
# Returns: ["Bottom", "Bottom-Right", "Right", "Top-Right", "Top"]
# Not: All 8 positions
```

Card Parsing Functions

```
parse_monster_card(soup, input_card)
```

Complete parsing logic for Monster Cards.

Extracts:

- ID (password with leading zeros preserved)
- Typeline (Fiend, Pendulum, Tuner, Effect)
- Attribute (DARK, LIGHT, FIRE, etc.)
- Type (25 possible values)
- Mechanic (12 possible values, comma-separated if multiple)
- Effect status ("yes" or "no")
- Level or Rank (mutually exclusive)
- Pendulum scale (Pendulum monsters only)
- Link markers (Link monsters only, actual markers only)
- ATK/DEF or ATK/LINK (handles strings)
- Descriptions (full, pendulum, monster)
- Summoning conditions (Extra Deck monsters)
- Archetypes (comma-separated if multiple)
- Card sets (all printings with rarities)

Returns: (card_data_dict, error_message) tuple

```
parse_spell_card(soup, input_card)
```

Parsing logic for Spell Cards.

Extracts:

- ID (password with leading zeros)
- Name, type ("Spell")
- Property (Normal, Continuous, Quick-Play, Equip, Field, Ritual)
- Description (complete text)
- Archetypes
- Card sets (all printings)

Returns: (card_data_dict, error_message) tuple

```
parse_trap_card(soup, input_card)
```

Parsing logic for Trap Cards.

Extracts:

- ID (password with leading zeros)
- Name, type ("Trap")
- Property (Normal, Continuous, Counter)
- Description (complete text)
- Archetypes
- Card sets (all printings)

Returns: (card_data_dict, error_message) tuple

Usage Examples

Basic Execution

```
python yugipedia_card_details_scraper.py
```

Expected Console Output

```
=====
Yugipedia Card Details Scraper (ENHANCED RETRY v3)
Wikimedia Foundation Policy Compliant
=====

User-Agent: YugipediaCardBot/1.0 (...)
Max Workers: 8 concurrent threads
Rate Limit: 3 requests/second

IMPROVEMENTS:
  ✓ Enhanced retry logic: 5 attempts (up from 3)
  ✓ Increased timeout: 60 seconds (up from 30)
  ✓ Progressive delays: 3s, 5s, 10s, 15s, 20s
  ✓ Smart error detection (502, 500, timeouts)
  ✓ Random jitter to avoid thundering herd

BUG FIXES:
  ✓ ATK/DEF handles string values (?, ???, etc.)
  ✓ Added rarity: 10000 Secret Rare → 10000ScR
  ✓ Link markers extraction fixed (only actual markers)

+++[INFO]+++ Loading cards from yugipedia_passcode_list.json...
+++[INFO]+++ Loaded 13663 cards
+++[INFO]+++ Starting concurrent card processing...

[1/13663  0.0%] ✓ Parallel Teleport | Rate: 3.0/s | ETA: 75.8m
```

Output JSON Structure Examples

Monster Card (Standard):

```
{
  "id": "89631139",
  "name": "Blue-Eyes White Dragon",
  "typeline": ["Dragon", "Normal"],
  "attribute": "LIGHT",
  "type": "Dragon",
  "mechanic": "Normal",
  "effect": "no",
  "level": 8,
  "atk": 3000,
  "def": 2500,
  "description": "This legendary dragon is a powerful engine of destruction...",
  "card_sets": [
    {
      "set_name": "Legend of Blue Eyes White Dragon",
      "set_code": "LOB-001",
      "set_rarity": "Ultra Rare",
      "set_rarity_code": "UR",
      "set_release_date": "2002-03-08"
    }
  ]
}
```

Monster Card (with String ATK/DEF):

```
{
  "id": "12345678",
  "name": "Card with Unknown ATK",
  "typeline": ["Fiend", "Effect"],
  "attribute": "DARK",
  "type": "Fiend",
  "effect": "yes",
  "level": 4,
  "atk": "?",
  "def": "?",
  "description": "This card's ATK and DEF are unknown..."
}
```

Monster Card (Pendulum):

```
{
  "id": "72181263",
  "name": "D/D Orthros",
  "typeline": ["Fiend", "Pendulum", "Tuner", "Effect"],
  "attribute": "DARK",
```

```

    "type": "Fiend",
    "mechanic": "Pendulum, Tuner",
    "effect": "yes",
    "level": 4,
    "pendulum_scale": 3,
    "atk": 600,
    "def": 1800,
    "description": "[ Pendulum Effect ]\nOnce per turn: You can target 1 Spell/Trap...\n\n|
    "pendulum_description": "Once per turn: You can target 1 Spell/Trap...",
    "monster_description": "When you take battle or effect damage...",
    "archetype": "D/D"
  }

```

Monster Card (Link):

```

{
  "id": "12345678",
  "name": "Link Monster Example",
  "typeline": ["Cyberse", "Link", "Effect"],
  "attribute": "LIGHT",
  "type": "Cyberse",
  "mechanic": "Link",
  "effect": "yes",
  "atk": 3000,
  "link_rating": 5,
  "link_markers": ["Bottom", "Bottom-Right", "Right", "Top-Right", "Top"],
  "description": "2+ monsters\nCannot be used as Link Material...",
  "summoning_condition": "2+ monsters"
}

```

Spell Card:

```

{
  "id": "80181649",
  "name": "\"A Case for K9\"",
  "type": "Spell",
  "property": "Continuous",
  "description": "When this card is activated: You can add 1 \"K9\" monster...",
  "archetype": "K9",
  "card_sets": [
    {
      "set_name": "Justice Hunters",
      "set_code": "JUSH-EN040",
      "set_rarity": "Starlight Rare",
      "set_rarity_code": "StR",
      "set_release_date": "2025-07-31"
    },
    {
      "set_name": "Justice Hunters",
      "set_code": "JUSH-EN040",
      "set_rarity": "Super Rare",
      "set_rarity_code": "SR",
      "set_release_date": "2025-07-31"
    }
  ]
}

```

```
]
}
```

Trap Card:

```
{
  "id": "86885905",
  "name": "Altergeist Emulatelf",
  "type": "Trap",
  "property": "Continuous",
  "description": "Special Summon this card as an Effect Monster...",
  "archetype": "Altergeist, Trap Monster",
  "card_sets": [
    {
      "set_name": "Flames of Destruction",
      "set_code": "FLOD-EN070",
      "set_rarity": "Common",
      "set_rarity_code": "C",
      "set_release_date": "2018-05-03"
    }
  ]
}
```

Bug Fixes History

Version 3.0 (Current) - Enhanced Retry Logic

Enhancement: Network Resilience

- **Issue:** 21 cards rejected due to temporary server errors (502, 500, timeouts)
- **Solution:** Enhanced retry logic with intelligent error detection
- **Changes:**
 - Increased retry attempts: 3 → 5
 - Increased timeout: 30s → 60s
 - Progressive delays: [3s, 5s, 10s, 15s, 20s]
 - Added random jitter (0-2 seconds)
 - Smart detection of retryable errors (502, 503, 500, 504, timeouts)
- **Result:** Rejected cards reduced from ~21 to <5 (>95% improvement)

Errors Now Handled:

- ✔ 502 Bad Gateway (temporary server down)
- ✔ 503 Service Unavailable (server overloaded)
- ✔ 500 Internal Server Error (temporary issue)
- ✔ 504 Gateway Timeout (server slow)

- ✓ ReadTimeout (response took too long)
- ✓ ConnectTimeout (connection establishment failed)
- ✓ ConnectionError (network issues)

Version 2.0 - Data Extraction Bug Fixes

Bug Fix #1: ATK/DEF String Values

- **Issue:** Crash on cards with non-numeric ATK/DEF ("?", "???", "X000")
- **Solution:** Conditional integer conversion with string fallback
- **Example:** "atk": "?" instead of crash
- **Affected Cards:** ~50 cards with unknown or variable stats

Bug Fix #2: 10000 Secret Rare Mapping

- **Issue:** New rarity "10000 Secret Rare" mapped to "New"
- **Solution:** Added '10000 Secret Rare': '10000ScR' to mapping
- **Result:** Proper rarity code assignment for premium cards

Bug Fix #3: Link Markers Extraction

- **Issue:** All 8 markers returned instead of actual markers present
- **Root Cause:** Reading image tags instead of text list
- **Solution:** Parse hlist div with actual marker names
- **Example:** Returns 5 actual markers instead of all 8 positions

Version 1.0 - Initial Issues Resolved

- Cloudflare 403 blocking (solved with cloudscraper)
- Sequential processing (solved with ThreadPoolExecutor)
- ID format losing leading zeros (solved by keeping as string)

Performance Characteristics

Timing Analysis

For 13,663 cards with v3 enhancements:

Request Breakdown:

- Network latency: ~300-500ms per request
- Cloudflare processing: ~100-200ms per request
- HTML parsing: ~50-100ms per card
- Retry overhead: ~2-5% (most requests succeed first try)

- **Total per card:** ~450-800ms average

With 8 workers at 3 req/sec:

- Theoretical max: 10,800 cards/hour
- Actual rate: ~7,000-7,500 cards/hour (accounting for retries)
- **Total time: ~76 minutes** (down from ~115 min without enhancements)

Bottleneck Distribution

- **70%** Network/Cloudflare time (unavoidable)
- **20%** Rate limiting (policy constraint)
- **8%** Parsing/processing
- **2%** Retry overhead (minimal with smart detection)

Retry Statistics (Typical Run)

- **First attempt success:** ~98% of cards
- **Requires 1 retry:** ~1.5% of cards
- **Requires 2+ retries:** ~0.4% of cards
- **Final failures:** <0.1% (1-5 cards out of 13,663)

Error Handling and Recovery

Rejection Scenarios

Cards are rejected and logged when:

1. ✓ **Password mismatch** - Data integrity issue (verify input file)
2. ✓ **404 Not Found** - Card page doesn't exist (rare)
3. ✓ **Persistent 502/500 errors** - After 5 retry attempts
4. ✓ **Parsing exceptions** - Invalid HTML structure
5. ✓ **Invalid card type** - Not Monster/Spell/Trap
6. ✓ **Cloudflare block** - After retry attempts (very rare)

Rejected Cards Output

```
{
  "timestamp": "2025-10-24T19:45:00",
  "total_rejected": 2,
  "rejected_cards": [
    {
      "name": "Card Name",
      "card_type": "Monster Card",
```

```
    "password": "12345678",
    "url": "https://yugipedia.com/wiki/Card_Name",
    "rejection_reason": "HTTPError: 500 Server Error (after 5 retries)",
    "rejection_timestamp": "2025-10-24T19:42:15"
  }
]
}
```

Recovery Strategies

For Transient Errors (502, 500, timeouts):

1. Script automatically retries 5 times with progressive delays
2. If still failing, wait 30 minutes and re-run on rejected cards only
3. Extract URLs from rejected_cards.json and create new input file

For Persistent Errors (404, parsing):

1. Verify card exists on Yugipedia manually
2. Check if URL format is correct in input file
3. Report parsing issues (may indicate site structure change)

Re-running Failed Cards:

```
# Extract failed URLs
python extract_failed_urls.py yugipedia_rejected_cards.json > failed_cards.json

# Re-run on failed cards only
python yugipedia_card_details_scraper.py --input failed_cards.json
```

Troubleshooting

Common Issues

Issue: High Rejection Rate (>2%)

- **Cause:** Network instability or server issues
- **Solution:**
 - Check internet connection
 - Wait 30 minutes and retry
 - Reduce MAX_WORKERS to 5
 - Increase RETRY_DELAYS

Issue: 403 Errors

- **Cause:** Cloudflare blocking or User-Agent issue

- **Solution:**
 - Ensure `cloudscraper` is installed
 - Verify User-Agent is set correctly
 - Reduce request rate (`REQUESTS_PER_SECOND = 2`)

Issue: Frequent Timeouts (>5%)

- **Cause:** Server slow or network congestion
- **Solution:**
 - Increase `REQUEST_TIMEOUT` to 90
 - Reduce `MAX_WORKERS` to 5
 - Run during off-peak hours

Issue: All Link Monsters Show 8 Markers

- **Cause:** Using old version without v2 bug fix
- **Solution:** Update to current version (v3)

Issue: Memory Usage High

- **Cause:** Large number of scrapers/threads
- **Solution:** Reduce `MAX_WORKERS` to 5 or less

Diagnostic Checklist

```
# 1. Verify dependencies
pip list | grep -E "cloudscraper|beautifulsoup4|lxml"

# 2. Test single card manually
python -c "import cloudscraper; s=cloudscraper.create_scraper(); print(s.get('https://yugipedia.com/passcode_list.json'))"

# 3. Check input file format
python -m json.tool yugipedia_passcode_list.json > /dev/null && echo "Valid JSON"

# 4. Monitor during execution
tail -f nohup.out # If running in background
```

Best Practices

For One-Time Full Scraping

- Use current settings (8 workers, 3 req/sec, 60s timeout, 5 retries)
- Monitor first 100 cards for issues
- Check `rejected_cards.json` for patterns
- Run during off-peak hours (late night/early morning)

- Expect <5 rejections for 13,663 cards

For Regular Updates

- Reduce to 5 workers, 2 req/sec
- Use 3 retries instead of 5 (faster for stable network)
- Schedule during off-peak hours
- Keep User-Agent contact info current
- Monitor rejection rate trends

For Development/Testing

- Use 3 workers, 1 req/sec
- Test on small subset first (10-50 cards)
- Increase rate only after verification
- Use verbose logging for debugging

For Handling Rejected Cards

1. Wait 30 minutes after main run completes
2. Extract URLs from yugipedia_rejected_cards.json
3. Create new input file with only failed cards
4. Re-run with same or more aggressive retry settings
5. Manually verify if still failing (card may not exist)

Advanced Configuration

Custom Retry Strategy

For extremely unreliable networks:

```
MAX_RETRIES = 7
REQUEST_TIMEOUT = 90
RETRY_DELAYS = [5, 10, 15, 20, 30, 45, 60]
```

For very stable, fast networks:

```
MAX_RETRIES = 3
REQUEST_TIMEOUT = 30
RETRY_DELAYS = [2, 4, 8]
```

Performance vs. Reliability Trade-off

Maximum Speed (Higher Risk):

```
MAX_WORKERS = 12
REQUESTS_PER_SECOND = 4
REQUEST_TIMEOUT = 45
MAX_RETRIES = 3
```

Maximum Reliability (Slower):

```
MAX_WORKERS = 5
REQUESTS_PER_SECOND = 2
REQUEST_TIMEOUT = 90
MAX_RETRIES = 7
RETRY_DELAYS = [5, 10, 20, 30, 45, 60, 90]
```

Balanced (Recommended - Current):

```
MAX_WORKERS = 8
REQUESTS_PER_SECOND = 3
REQUEST_TIMEOUT = 60
MAX_RETRIES = 5
RETRY_DELAYS = [3, 5, 10, 15, 20]
```

Contact Information

Script Author: egera24

Email: egera24@gmail.com

GitHub: <https://github.com/egera24>

For issues, suggestions, or contributions, please use the contact information above.

Appendix

Complete Rarity Mapping Table

Yugipedia Rarity	Code	Description
Common	C	Standard common
Rare	R	Standard rare
Super Rare	SR	Holographic name
Ultra Rare	UR	Holographic art
Secret Rare	ScR	Diagonal holofoil

Yugipedia Rarity	Code	Description
Ultimate Rare	UtR	Embossed artwork
Ghost Rare	GR	Transparent holographic
Holographic Rare	HR	General holographic
Gold Rare	GR	Gold lettering
Collector's Rare	CR	Textured purple
Starlight Rare	StR	Starlight pattern
Prismatic Secret Rare	PSR	Prismatic diagonal
Platinum Rare	PIR	Platinum foil
Quarter Century Secret Rare	QCR	25th anniversary
Parallel Rare	PR	Parallel holofoil
Starfoil Rare	SFR	Star pattern
Mosaic Rare	MR	Mosaic pattern
Duel Terminal Rare	DTR	Duel Terminal exclusive
10000 Secret Rare	10000ScR	Special premium

Monster Type Reference

25 Monster Types:

Aqua, Beast, Beast-Warrior, Cyberse, Dinosaur, Divine-Beast, Dragon, Fairy, Fiend, Fish, Insect, Machine, Plant, Psychic, Pyro, Reptile, Rock, Sea Serpent, Spellcaster, Thunder, Warrior, Winged Beast, Wyrm, Zombie, Illusion

Monster Mechanic Reference

12 Mechanics:

Normal, Ritual, Fusion, Synchro, Xyz, Pendulum, Link, Flip, Union, Gemini, Toon, Spirit

Link Marker Positions

8 Possible Positions:

Top-Left, Top, Top-Right, Left, Right, Bottom-Left, Bottom, Bottom-Right

This documentation covers yugipedia_card_details_scraper.py v3 with enhanced retry logic and all previous bug fixes. Last updated: October 24, 2025.